Use Case: Establish connection

Iteration: 1

Primary Actor: Player

Goal in Context: To establish a secure and stable connection with the network to access game services

Preconditions: Player has launched the game, Network Services are online and available

Trigger: Player initiates a connection attempt when starting the game

Scenario:

1. Player launches the game and reaches the main menu
2. Player selects an option that requires an online connection (all three of our games need it)
3. `NetworkManager` receives the player's request to connect
4. `NetworkManager` verifies player credentials and initiates a secure session (token? authentication method?)
5. `NetworkManager` checks the network quality and stable connection
6. Player is now connected to the network

Post Conditions: Player is successfully connected and ready to access online features (leaderboard, SBMM, chat)

Exceptions: Network is down and unavailable. Connection timeout: the network takes too long to respond.Poor connection quality.

Priority: High

When Available: 2nd iteration

Frequency of use: Every time a player initiates a connection or re-connects

Channel to Actor: GUI

Secondary Actor: Network (mostly `NetworkManager`)

Channels to Secondary Actor: Stub? (realistically an API call)

Open Issues: Say a player keeps having to reconnect, should there be a timer to stage intervals?

Use Case: Send Message

Iteration: 1

Actor: Player

Goal in Context: To allow a player to send a message to their opponent during a game session

Preconditions: Both players are connected to the same game session chat functionality is enabled for the game session

Trigger: Player initiates a message by typing into the in-game chat interface

Scenario:

1. Player opens the in-game chat interface
2. Player types a message in a text input box
3. Player sends their message by clicking "Send" button or pressing "Enter" on their keyboard.
4. `Network Manager` receives the message and forwards it to `ChatManager.`
5. `ChatManager` routes the message to the opponent
6. The message appears in the opponent's chat interface

Post Conditions: The message is received and displayed in the opponent's chat interface/window both players can see the updated chat log with the new message

Exceptions: A player disconnecting while in the process of sending their message. Network failure. Message blocking (inappropriate language).

Priority: High

When Available: 2nd iteration

Frequency of use: It varies as it depends on a player's communication preferences

Channel to Actor: In-game chat interface/window

Secondary Actor: Network - `NetworkManager` and `ChatManager`

Channels to Secondary Actor: N/A

Open Issues: Filtering chat which can be a long and heavy process

Use Case: Queue Entry

Iteration: 1

Actor: Player

Goal in Context: To allow a player to enter a matchmaking queue and wait for a compatible opponent to begin a game session

Preconditions: Both players are connected to the network. Matchmaking services are active and available

Trigger: Player selects an option to queue up matchmaking

Scenario:

1. Player navigates to the three different games they want to play in the GUI
2. Player chooses a game by selecting one
3. Player chooses to join matchmaking queue by clicking "find game" button in the GUI
4. `NetworkManager` sends a request to `MatchmakingService` to add the player to the queue
5. `MatchmakingService` places the player in a queue and begins searching for the right opponent based on SBMM criteria
6. Player receives a notification/status update to notify that they have found an opponent
7. The player is transitioned to the game session with their matched opponent

Post Conditions: Player is currently in an active match with their opponent or still remains in the queue if no match is available

Exceptions: Network failure. Timeout connection - if a player can't find a match within a certain amount of time. One of the player disconnects or leaves the matchmaking queue. Too many players in the matchmaking queue - a capacity limit

Priority: High

When Available: 2nd iteration

Frequency of use: Frequent as our games are all based on multiplayer

Channel to Actor: GUI interface

Secondary Actor: Network - `MatchmakingService`

Channels to Secondary Actor: Stub (API if we're being serious) calls and notifies from `NetworkManager` to `MatchmakingService`

Open Issues: We should define a time limit when waiting in queue before a timeout

Use Case: Status Update

Actor: Player

Goal in Context: To provide a player with updates regarding their current status within the game. For example, connection quality and matchmaking queue position

Preconditions: player is connected to network and using the network features - SBMM, game chat, etc... `NetworkManager` or `MatchmakingService` is actively tracking player status

Trigger: A change in a player's status or an update from the network that needs to be communicated to the player

Scenario:

1. Player initiates an action that requires a status update - entering matchmaking queue or sending a message
2. `NetworkManager` detects a change in the payer's current activity - moving up a spot in matchmaking queue
3. `NetworkManager` sends a status update to the player's GUI/interface regarding their current activity - estimated wait time, current queue position, network status etc...
4. Player receives and views the update on their screen
5. Player may adjust their actions - exit queue if it's taking too long, latency issues / connection status

Post Conditions: The player is aware of their current status and makes a decision or not based on the updated information

Exceptions: Network failure. A player may disconnect before the status update is

received. Network latency might cause delay to status updates - potentially giving outdated or slow feedback.

Priority: Medium - it gives a nice experience for the players to know what their status is, but not as important as gameplay functionality

When Available: 3rd iteration

Frequency of use: It varies as it'll be based on network events or player actions

Channel to Actor: GUI - notification or a status window

Secondary Actor: Network - `NetworkManager` and `MatchmakingService`

Channels to Secondary Actor: Stub (API) calls for status changes

Open Issues: We have to determine which relevant statuses to update the player on the frequency and method of update delivery

Use Case: Accept/Reject Match

Primary Actor: Player

Goal in Context: Allows the player to accept/reject the match that has been found based on the matchmaking system.

Preconditions:

The player is already in the matchmaking queue.

Matchmaking is successful and returns a suitable match as per the player's stats.

Both the players are connected to the server

Trigger: The matchmaking system successfully finds a suitable player and informs both the players about the match that has been found.

Scenario:

1. Player has selected the game that he wants to play.
2. The player is then put on the queue if there are multiple players playing and in the queue. The matchmaking system now starts and finds suitable opponent for the player.
3. Both the players are displayed the matches.
4. The system notifies each player about the matches and waits for the

response from the player. The player must either accept/reject. If rejected the player is put on queue and waits for the next match. If both the player accepts the match, the players are transitioned into game screen and the game begins.

Postconditions:

If player accepts the match found, then the game begins.

If player rejects the match, then the player is put on the matchmaking queue and waits until they are matched with the next player.

Exceptions:

The player doesn't respond on time when prompted with accept/reject match. If either of the players has an unstable connection.

Game is not loaded/created after the match is accepted.

Priority: High

When Available: 3rd Iteration

Frequency of Use: Very frequent (depending on the number of players playing at a time)

Channel to actor: Player interacts with the UI using mouse click for selecting the accept/reject button.

Use Case: Game Move

Primary Actor: Player

Goal in Context: Enables the user to execute their move in the game and also maintaining the proper game state synchronization between the two players.

Preconditions:

Player is matched with their suitable opponent.

The game starts and it is the player's turn to make their move.

The game state is properly syncronized and reflects the same at the other

end on the player's side. Both the players are active and there no disruption of connection between them.

Trigger: Player registers their move in the game and plays the turn.

Scenario:

1. The player has started the game
2. The game notifies the player it's their turn and is waiting for the player's move.
3. Player plays their and register's it's move.
4. The system then checks if the move made was valid or not.
5. After the check is done, the move is then registered and made visible to both the players. The game state is updated and synchronized properly.

Postconditions:

The game state is updated and maintained

After the move:

If the move made by the player results in a win/loss/draw, the game is over and the state is updated accordingly.

Otherwise the turn then shifts to the opponent.

Exceptions:

Invalid/Not accepted move is played.

Delay in response thereby affecting the game state and synchronization.

Either of the player gets disconnected while playing their move.

Priority: High

When Available: 3rd Iteration

Frequency of Use: Very frequent (as this will allow the users to play eventually)

Channel to actor: Player plays the game with just simple mouse clicks/keys as per the game functionality.

Use Case: End game

Primary Actor: Network

Goal in Context: Terminates a game session, whenever a winning move is played thereby resulting the game to end and then notifies both the players about the results.

Preconditions:

Both the players are in an active game session.

Either of the players have played a winning move resulting in game termination.

Either of the players have played a move that resulted in a draw, the game also terminates in that case and notifes the player.

Either of the players have forfeited. A forefiet will terminate the game and the player who forfeited will automatically lose.

Trigger: The system detects that either of the players has played a winning/losing/drawing move.

Scenario:

1. The system receives the last move from the player.
2. The move is then verified and checked.
3. The game state is updated and synchronized for both the players.
4. The system then does all the calculations and detects that it is a winning move played by either of the players.
5. The system then displays the game results to both the players and indicates that the game is over. The system then updates the leaderboard as per the game results and player's stats.

Postconditions:

The game session is terminated properly.

Player rankings are updated accordingly in the leaderboard after

every game played.

Exceptions:

Connection disruption may occur due to which even after playing a winning move, the system couldn't detect that and the game doesn't end.

The move played wasn't checked properly might also lead to end the game abruptly.

The game stats for the players weren't updated properly.

Priority: High

When Available: 3rd Iteration

Frequency of Use: Very frequent (after every game session)

Channel to actor: The system coordinates the game ending process automatically after it detects the move played.

Use Case: View Leaderboard

Primary Actor: Player

Goal in Context: Allows player to check the leaderboard in order to view their current standings and rankings for other players.

Preconditions:

The player is logged in and has played sufficient number of games in order to be visible on the leaderboard.

Leaderboard is available to retrieve the rankings for all players as per their gaming history and stats.

Trigger: Whenever the player requests to view the leaderboard.

Scenario:

1. The player has finished their game session.
2. The player then requests to view the leaderboard.
3. The request is received by the system and thereby it fetches all the current top rankings from the database.

4. The GUI interface receives the requested data and displays it as a leaderboard.
5. The players therefore can then view their current standings and compare it with their opponents too.

Postconditions:

The player is able to successfully retrive the leaderboard data and view their rankings. The rankings calculated are correct and that it was updated as per the latest game played by the player.

Exceptions:

There was a disruption in connection which caused miscalculation in rankings thereby the ranking displayed was incorrect.

The leaderboard rankings were not updated after each game played.

Priority: High

When Available: 3rd iteration

Frequency of Use: Quite a few sometimes.(Depends on the player)

Channel to actor: Player interacts with the GUI just normally using the mouse in order to navigate and view the leaderboard.